

NexusTriage Project Documentation

1. System Overview

NexusTriage is a multi-channel triage and automation system that integrates with Gmail, WhatsApp, and Telegram. It utilizes intelligent routing, AI-based analysis, and automated workflows to manage and escalate user communications effectively.

This documentation covers the backend workflows powered by **n8n**, the containerized infrastructure, the database, the onboarding process, the dashboard, and the local AI processing using **Ollama**.

2. Infrastructure & Docker Container Setup

The project relies on a containerized architecture orchestrated by Docker Compose (`docker-compose.yml`), ensuring consistency and easy deployment.

Services:

- **postgres**: Runs PostgreSQL 15 as the core database. The database is initialized via scripts in the `supabase` directory.
 - **redis**: Runs Redis 7 (Alpine), acting as a fast caching layer and message broker for background services.
 - **evolution-api**: An instance of Evolution API (v2.1.1) that handles WhatsApp connections. It connects to Postgres and Redis and exposes a global webhook pointing to n8n (`http://n8n:5678/webhook/whatsapp-incoming`) for incoming WhatsApp events.
 - **n8n**: The core workflow automation engine (version 1.60.0). It is connected to the Postgres database for data persistence and acts as the orchestrator for incoming webhooks and scheduled tasks.
 - **nexustriage-api**: The custom backend service built from `./nexustriage-server` , which communicates with Evolution API, the n8n workflows, and the PostgreSQL database.
-

3. n8n Workflows

The automation logic is defined in several n8n workflow JSON files, which handle different facets of the system.

3.1 SaaS Master Workflow (`master_workflow.json`)

This workflow handles incoming webhooks from the Evolution API (WhatsApp messages).

- It queries the `users` and `keywords` tables in PostgreSQL.
- It forwards the incoming message to an LLM for triage analysis.
- Based on the intent (e.g., "Escalation"), it routes the message.
- For escalations, it inserts a pending record into the `escalations` table and sends an interactive approval request via Telegram.

3.2 Scheduled Escalation Digest (`workflow_escalation_digest.json`)

This workflow operates on a cron schedule (`0 17 * * *` - daily at 5:00 PM).

- It retrieves all `Pending` escalations from the database.
- If pending escalations exist, it aggregates them into a comprehensive daily digest.
- The digest is sent via Telegram to the relevant user.
- Finally, it marks the escalated records as `Sent` in the database.

3.3 Gmail Extractor & AI Drafter (`workflow_gmail_drafter.json`)

This workflow polls Gmail every minute for unread emails.

- It leverages the AI engine to analyze the email content and generate a short summary and sentiment analysis.
 - It generates a professional email reply draft using the LLM.
 - The workflow creates the draft directly in Gmail, linked to the original email thread.
 - The processed email content is logged into the `corpus` table in PostgreSQL for tracking.
-

4. Supabase (Database) Integration

The `supabase` directory manages the PostgreSQL database structure, which serves as the central source of truth for the system. The schema includes critical tables:

- `users` : Stores user configurations, WhatsApp session IDs, Telegram Chat IDs, etc.
 - `keywords` : Stores user-defined keywords to filter relevant communications.
 - `escalations` : Tracks escalated messages, capturing the sender, message content, and status (e.g., Pending, Sent).
 - `corpus` : Logs processed email contents and AI summaries for historical analysis.
-

5. Local AI Processing with Ollama

While the n8n JSON workflows can connect to cloud LLMs (like Groq), the core project also supports robust **local AI processing** via **Ollama**, ensuring complete data privacy and security.

The system is configured to run the **Phi-4 Mini (k_m q4)** model locally on port 11434 . This model is specifically utilized by the intelligent agents to perform triage, generate escalation digests, and draft emails without sending sensitive data to external servers. The integration is achieved by making HTTP POST requests to `http://localhost:11434/api/generate` with carefully crafted payloads, instructing the Phi-4 model to analyze intent, summarize, or mirror communication styles securely and efficiently.

6. Onboarding & Dashboard UI

The user interface (`NexusTriage_Onboarding_UI`) is built with React and Vite. It features an intuitive flow to set up all necessary integrations.

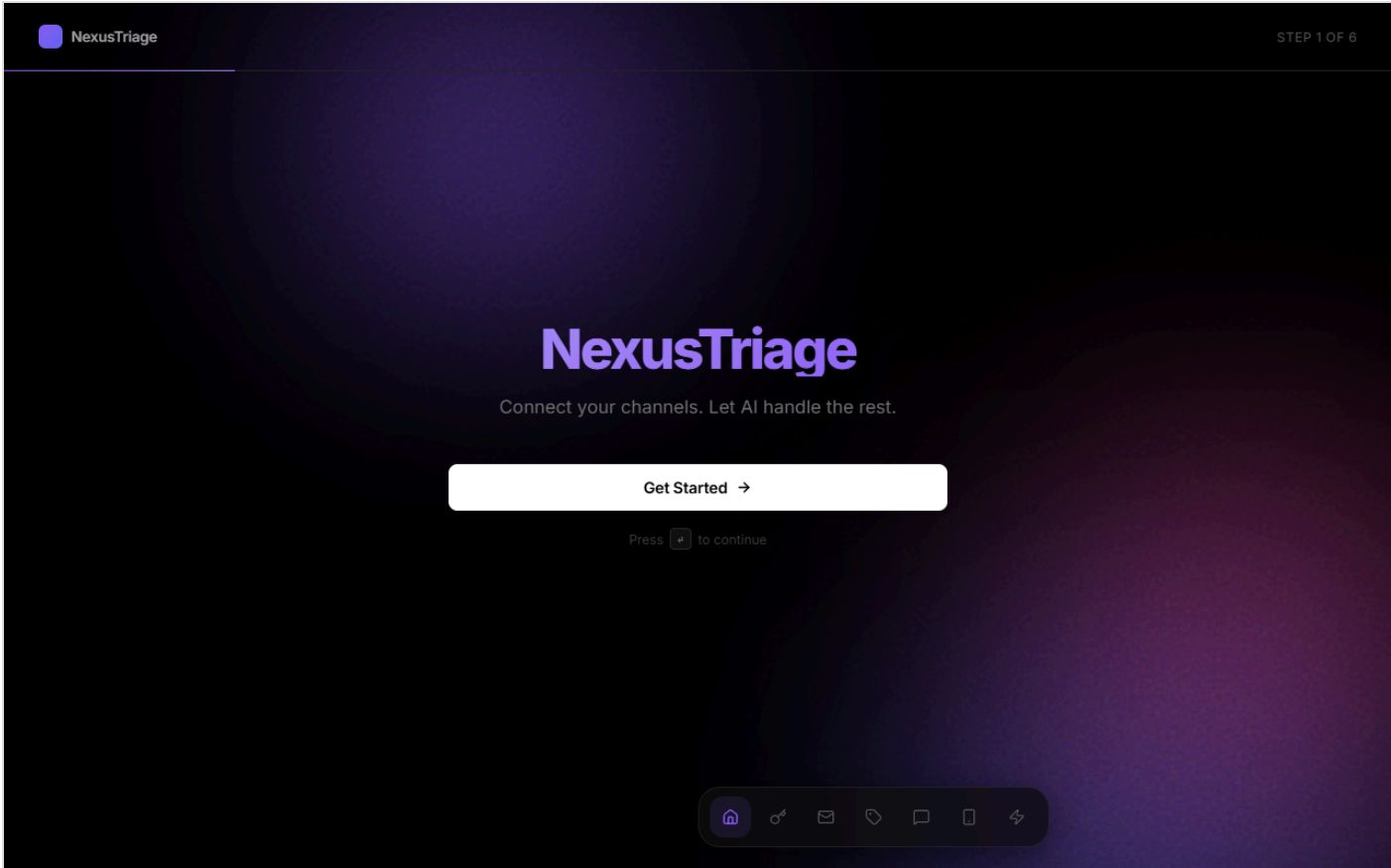
6.1 Onboarding Steps

The Onboarding Wizard guides new users through a 7-step process:

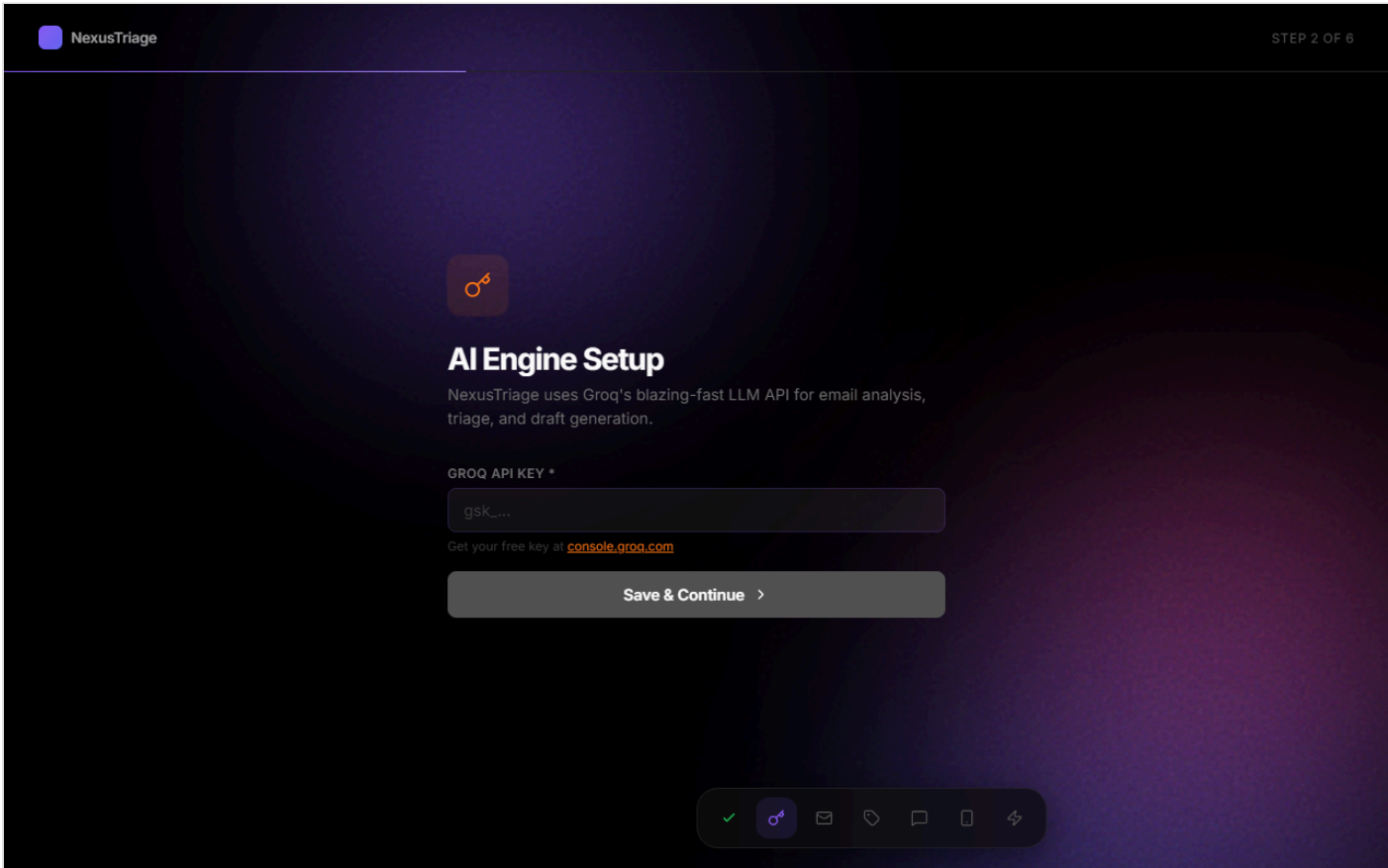
1. **Welcome**: Introduction to the platform.
2. **API Keys**: Configuration of the AI engine credentials.
3. **Gmail**: Google OAuth setup to allow the system to securely read and draft emails.
4. **Keywords**: Definition of trigger words (e.g., "invoice", "urgent") for the AI to monitor.
5. **Telegram**: Configuration of the Telegram Bot Token and Chat ID for receiving notifications and approvals.
6. **WhatsApp**: Integration with Evolution API using a dynamic QR code scanner.
7. **Done**: Final confirmation, leading directly to the Dashboard.

6.2 Snippets

Onboarding Wizard - Step 1:



Onboarding Wizard - Step 2:



Dashboard:

NexusTriage

Postgresn8nWhatsAppRefreshSettings

Run Email Scan Now

EmailsWhatsAppEscalationsDrafts

No emails analyzed yet. Add keywords and run a scan.